FLAPPY BIRD CLONE — TEST CASES

PS No. 61 | Python 3 | Pygame | Arranged in the same order as the assignment

Project specifications: Window 400 × 600; bird 30 × 30 at x=100, y=300; initial velocity=0; gravity update is velocity += 1 then y += velocity; SPACE sets velocity=-10 before update; pipe width=60, gap=150; ground y=600.

Test Case 1: Normal Input

Step	Input / Action	Expected Result
1	Initialize bird: y=300, velocity=0.	Bird starts at y=300 with velocity=0.
2	No key pressed; run one frame.	Frame 1: velocity=1, y=301.
3	No key pressed; run second frame.	Frame 2: velocity=2, y=303.

Test Case 2: Boundary / Minimum Input

Subcase	Input / Action	Expected Result
(a) Ground boundary	Set bird y=570 (30 px high); bottom edge reaches y=600.	Ground collision detected; Game Over.
(b) Top boundary	Set bird y=5, velocity=0; press SPACE; update one frame.	SPACE sets velocity=-10; frame update: velocity=-9, y=-4. Bird leaves top; Game Over.

Test Case 3: Invalid Input

Step	Input / Action	Expected Result
1	Initialize bird y=300, velocity=0.	Bird is at y=300; velocity=0.
2	Press Z (not SPACE); run one frame.	Invalid key is ignored. Frame update gives velocity=1, y=301.

Test Case 4: Different Valid Scenario

Subcase	Input / Action	Expected Result
(a) Flap with positive velocity	Set bird y=300, velocity=5; press SPACE; run one frame.	SPACE resets velocity=-10; after frame update velocity=-9, y=291. Bird moves upward.
(b) Pipe passing / scoring	Bird x=100; pipe x=39, width=60; score=0.	Pipe right edge = 99, which has passed bird x=100; pipe passed and score becomes 1.

Pass criteria: The program must produce the expected velocity, position, collision/game-over state, and score shown above for each test. The frame update order must remain: (1) apply SPACE velocity if pressed, (2) velocity += 1, (3) y += velocity.